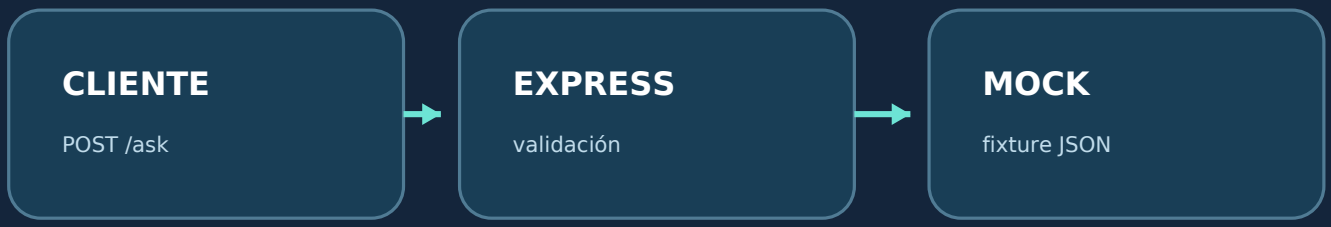
GUÍA VISUAL · MÓDULO 0

# De la pregunta del cliente a la respuesta MOCK

Arquitectura, responsabilidades de carpetas y archivos, recorrido de una solicitud HTTP y respuesta simulada sin utilizar un LLM remoto.




## Idea central

En modo mock no hay llamada a OpenRouter, Ollama ni consumo de tokens. La función de inferencia carga `fixtures/mock-response.json` y retorna un objeto tipado que Express entrega al cliente.

### Proyecto integrador: AI Knowledge Assistant

Tecnologías del módulo: Node.js · TypeScript estricto · Express 5 · Zod · fixtures JSON · Vitest y Supertest.

---

Material para explicación en clase y sustentación técnica

2026

DIAGRAMA 1

# Arquitectura y responsabilidades

Arranque del servidor, validación de entrada, bifurcación y respuesta HTTP

## MÓDULO 0 · De la pregunta del cliente a la respuesta MOCK

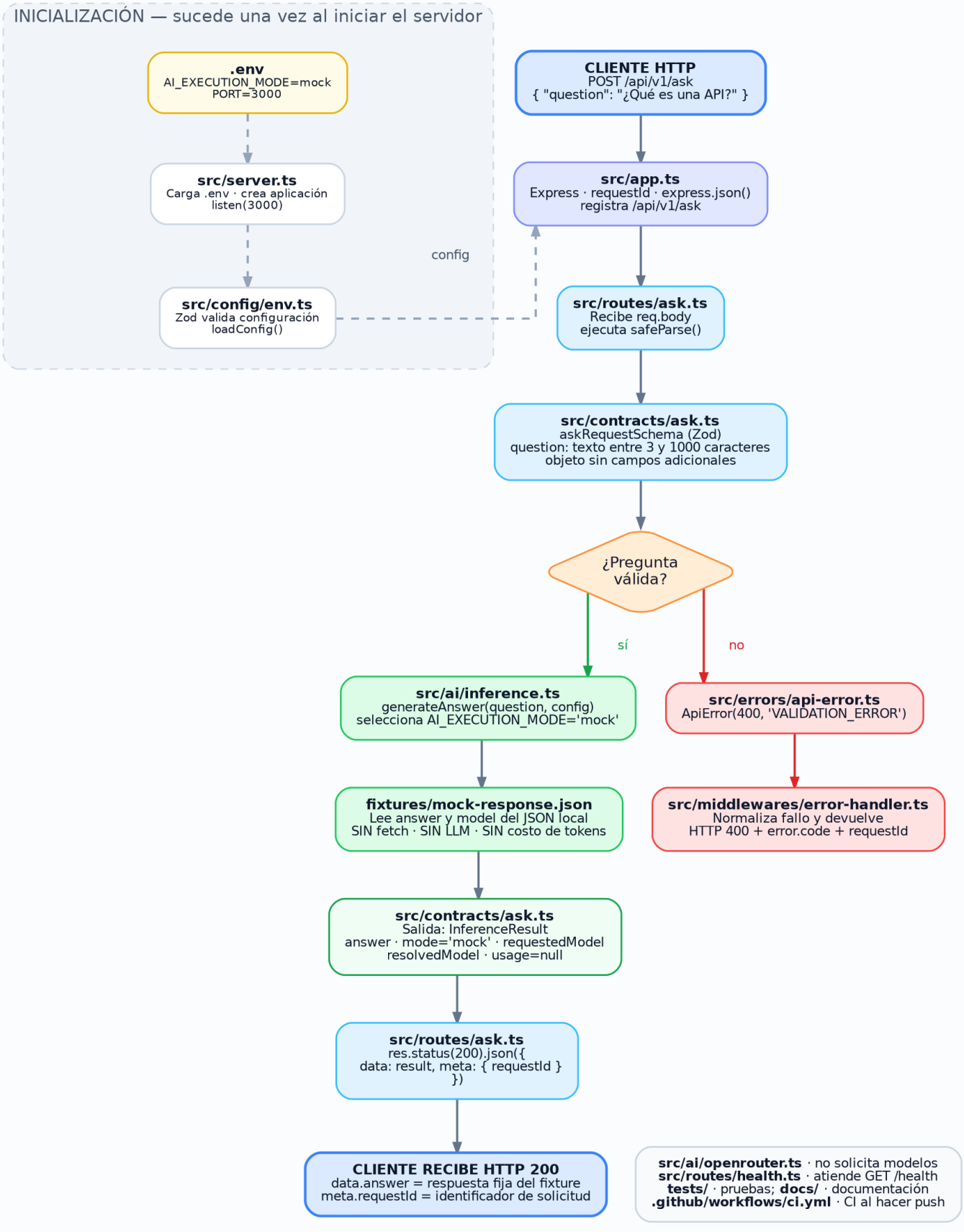
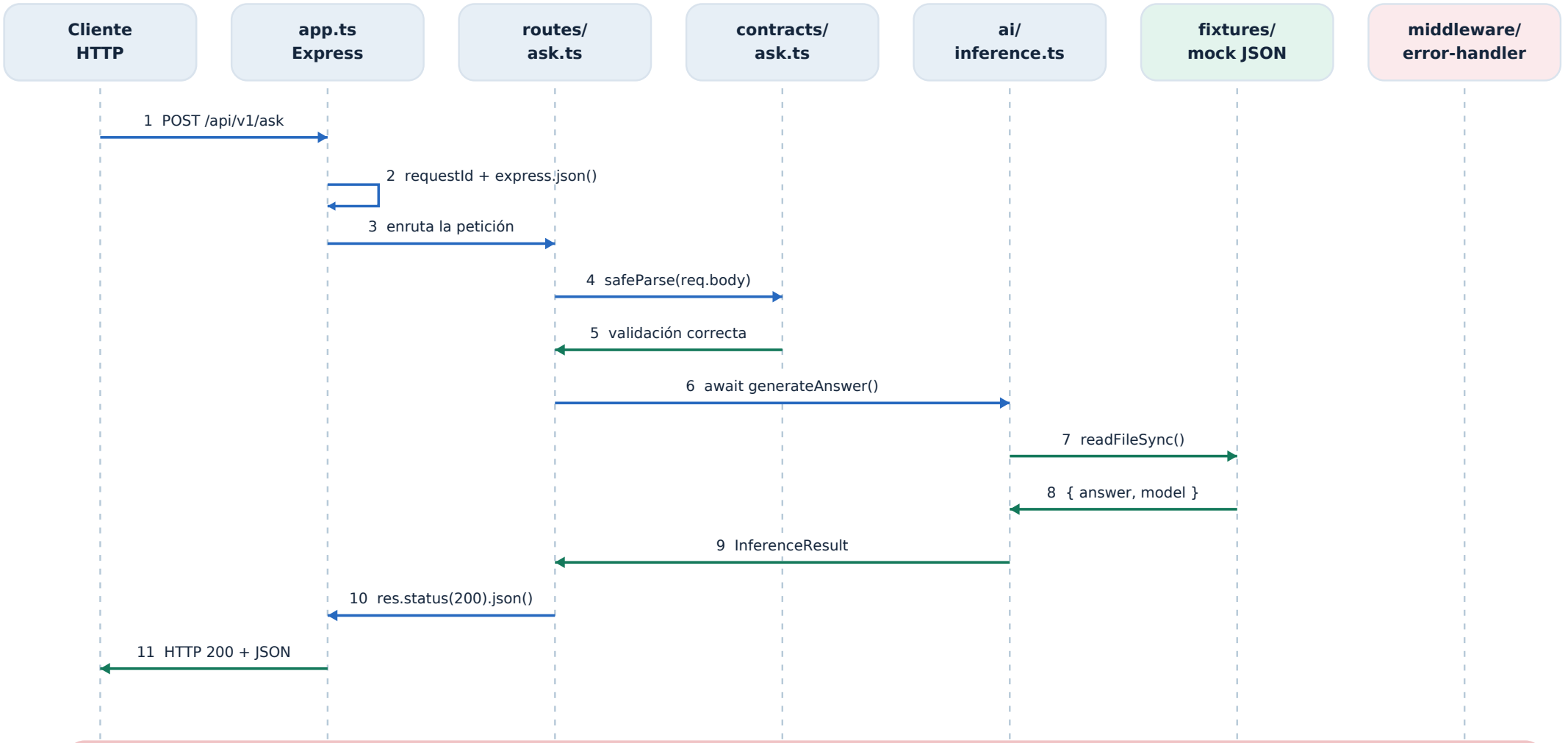


DIAGRAMA 2

# Secuencia de ejecución: solicitud válida

Las líneas verticales representan las responsabilidades que intervienen en el flujo




**RUTA ALTERNATIVA · SI LA VALIDACIÓN FALLA**

routes/ask.ts lanza ApiError(400) → error-handler.ts normaliza → el cliente recibe HTTP 400 + error.code + requestId

EN MODO MOCK: no existe tráfico de inferencia hacia OpenRouter ni hacia Ollama.

GUÍA DE LECTURA

# ¿Qué hace cada carpeta y archivo?

Separación de responsabilidades y ejecución real durante una pregunta

| Capa o carpeta | Archivo | Responsabilidad |
|---|---|---|
| **Configuración / raíz** | .env | Fija AI_EXECUTION_MODE=mock y PORT=3000. |
| **Arranque** | src/server.ts | Carga .env, construye app y abre listen(3000). |
| **Configuración** | src/config/env.ts | Valida variables mediante Zod (loadConfig). |
| **HTTP** | src/app.ts | Registra express.json(), requestId y routers. |
| **Rutas** | src/routes/ask.ts | Procesa POST /ask, valida y devuelve JSON. |
| **Contratos** | src/contracts/ask.ts | Define askRequestSchema e InferenceResult. |
| **Inferencia** | src/ai/inference.ts | Selecciona modo mock y carga fixture JSON. |
| **Fixture** | fixtures/mock-response.json | Almacena answer y model de respuesta fija. |
| **Errores** | src/errors/api-error.ts | Define error HTTP con statusCode y code. |
| **Middleware** | src/middlewares/error-handler.ts | Convierte fallas en JSON de error estable. |
| **Otros** | src/ai/openrouter.ts | NO se invoca con AI_EXECUTION_MODE=mock. |
| **Otros** | src/routes/health.ts | Sirve GET /health, independiente de /ask. |
| **Calidad** | tests/, docs/, .github/ | Pruebas, documentación y CI; no intervienen en la pregunta. |

## 1 Dos fases distintas

**Inicialización (una vez al arrancar)**

server.ts → loadConfig() → createApp() → registro de rutas → listen().

**Petición HTTP (por cada pregunta)**

app.ts → routes/ask.ts → Zod → generateAnswer() → fixture → HTTP 200.

LABORATORIO

# Ejemplo práctico y comprobaciones

La respuesta mock es fija: no depende semánticamente de la pregunta

## 1 El cliente envía una pregunta

```
POST http://127.0.0.1:3000/api/v1/ask
Content-Type: application/json

{"question":"¿Qué es una API REST?"}
```

## 2 La función lee el fixture local

```
fixtures/mock-response.json
{
  "model": "mock/fixture-v1",
  "answer": "Una API conecta aplicaciones mediante solicitudes y respuestas HTTP."
}
```

## 3 Express devuelve la respuesta

```
HTTP/1.1 200 OK
{
  "data": {
    "answer": "Una API conecta aplicaciones ...",
    "mode": "mock",
    "requestedModel": "mock/fixture-v1",
    "resolvedModel": "mock/fixture-v1",
    "usage": null
  },
  "meta": { "requestId": "<UUID de la petición>" }
}
```

**EVIDENCIA DE APRENDIZAJE**

Explica por qué ask.ts no lee el archivo JSON, qué valida safeParse() y por qué fetch() no debe ejecutarse en modo mock. Después prueba una pregunta vacía y justifica el HTTP 400.